RBE 577: Machine Learning for Robotics Project 4: Imitation Learning Expert

Filippo Marcantoni
Robotics Engineering, WPI
fmarcantoni@wpi.edu

Prahladh Raja
Robotics Engineering, WPI
pnamboorkrishnam@wpi.edu

***Abstract*—This paper presents an experimental study of imitation learning for robotic manipulation using robosuite and robomimic. We collected 59 successful expert demonstrations for the `PickPlaceCan` task using a Panda robot and trained both Behavioral Cloning (BC) and Diffusion Policy models. The initial feedforward BC baseline achieved a rollout success rate of only 0.02. After systematic hyperparameter tuning, most importantly by introducing temporal context through an LSTM with sequence length 20, the best BC configuration achieved a success rate of 0.80. Multiple Diffusion Policy configurations were also trained and evaluated. The best diffusion configurations were image-based and achieved a success rate of 0.50. Overall, the recurrent BC model outperformed Diffusion Policy on this dataset, likely because the dataset size was relatively small and the low-dimensional recurrent policy was easier to optimize than the higher-capacity diffusion model. These experiments demonstrate that sequence length, model architecture, learning rate, training duration, observation modality, and prediction horizon all significantly affect imitation learning performance for contact-rich manipulation tasks.**

I. Introduction

The goal of this project was to train imitation learning policies for robotic manipulation using expert demonstrations. We used robosuite [1] to collect demonstrations and robomimic [2] to train the resulting policies. The selected task was `PickPlaceCan`, in which a Panda robot must pick up a can from a table and place it into a designated target bin.

Two imitation learning paradigms were evaluated. *Behavioral Cloning* (BC) treats policy learning as a supervised regression problem: the policy directly predicts expert actions from observations and is trained to minimize the mean-squared error between predicted and demonstrated actions. *Diffusion Policy* models action generation as an iterative conditional denoising process, producing action sequences from Gaussian noise conditioned on recent observations.

The objectives of this project were to:

- collect a dataset of at least 50 successful expert demonstrations;
- convert the dataset into the robomimic HDF5 format and generate image observations from simulator states;
- train and systematically tune Behavioral Cloning policies, including feedforward MLP, GMM, and recurrent LSTM variants;
- train and tune Diffusion Policy models using both low-dimensional and image-based observation modalities;
- compare final performance using rollout success rate, validation loss, training stability, rollout videos, and sensitivity to hyperparameters.

The remainder of this paper is structured as follows. Section II describes the task and environment. Section III covers data collection and processing. Section IV presents the two imitation learning formulations. Sections V and VI report the experimental results for BC and Diffusion Policy, respectively. Section VIII compares the two methods. Section **??** discusses the effect of demonstration count and the limitations of the current experiments. Section IX discusses practical challenges, and Section X concludes the report.

II. Task and Environment Description

A. Task Overview

The selected environment was `PickPlaceCan`, a standard robosuite benchmark for contact-rich manipulation. In this task, the robot must: (i) approach and grasp a can resting on a tray; (ii) lift the can; (iii) transport it to the target bin; and (iv) release it successfully. The task is sequential, and failure at any phase typically cascades into an unsuccessful rollout. For example, if the robot grasps the can poorly, later transport and placement actions cannot recover the task.

The experimental configuration is summarized in Table I.

TABLE I
Environment configuration.

Parameter	Value
Environment	`PickPlaceCan`
Robot	Panda
Simulator	MuJoCo via robosuite
Control interface	Keyboard teleoperation
Training framework	robomimic
Camera views	`agentview`, `robot0_eye_in_hand`
Image resolution	84×84 pixels
Dataset size	59 successful demonstrations

B. Environment Visualization

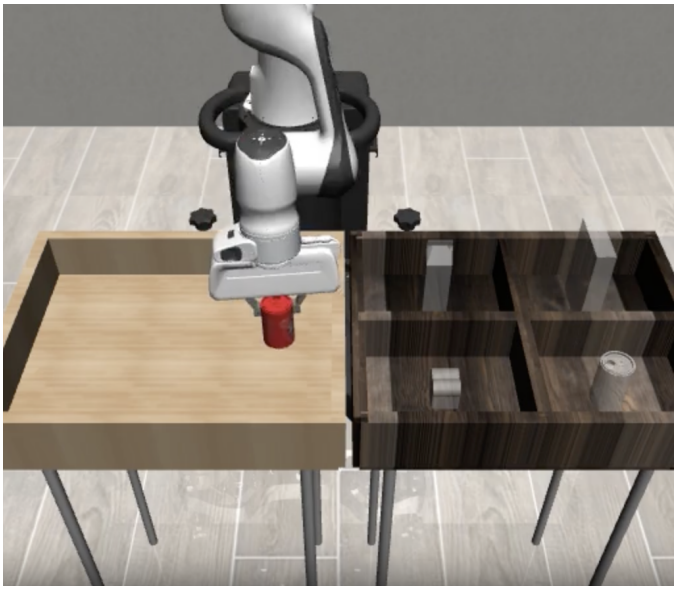

Fig. 1. The `PickPlaceCan` task. A Panda robot must pick up the can and place it into the correct target bin.

III. Data Collection and Dataset Processing

A. Demonstration Collection

We collected **59 successful demonstrations** using the robosuite keyboard teleoperation interface. Demonstrations were collected locally because real-time rendering and keyboard control were more reliable on a local workstation than on the Turing Cluster login nodes. Only trajectories that successfully completed the pick-and-place task were retained for training.

Before collecting the full dataset, we practiced the keyboard controls to generate smooth and consistent trajectories. Particular attention was given to the grasping phase, since an imprecise grasp often caused later placement failure.

B. Dataset Conversion

After collection, the raw robosuite HDF5 file was transferred to the WPI Turing Cluster. The following processing pipeline was applied:

1) **Format conversion:** `convert_robosuite.py` converted the raw robosuite demonstrations into the robomimic HDF5 schema.
2) **Train/validation split:** `split_train_val.py` created train and validation masks in the HDF5 dataset. The split used approximately 70% of the demonstrations for training and 30% for validation.
3) **Observation rendering:** `dataset_states_to_obs.py` replayed each trajectory in MuJoCo and rendered RGB observations from `agentview` and `robot0_eye_in_hand` at 84×84 resolution, producing the final `demo_obs.hdf5` dataset.

C. Observation Keys

The low-dimensional observation keys used throughout the experiments were:

- `robot0_eef_pos`: end-effector Cartesian position;
- `robot0_eef_quat`: end-effector orientation quaternion;
- `robot0_gripper_qpos`: gripper joint positions;
- `object`: object-related state, including the can state.

The RGB image observation keys available in the rendered dataset were:

- `agentview_image`;
- `robot0_eye_in_hand_image`.

IV. Methods

A. Behavioral Cloning

Behavioral Cloning (BC) learns a policy π_θ by minimizing the discrepancy between the policy's predicted action and the expert action. For a feedforward policy:

$$\hat{a}_t = \pi_\theta(o_t). \tag{1}$$

The training objective is the mean-squared error over the demonstration dataset $\mathcal{D}$:

$$\mathcal{L}_{\mathrm{BC}} = \mathbb{E}_{(o_t,a_t)\sim\mathcal{D}}\left[\|\pi_\theta(o_t) - a_t\|_2^2\right]. \tag{2}$$

BC is simple and efficient, but it is susceptible to covariate shift: small prediction errors accumulate during closed-loop execution and can drive the policy into states not present in the expert demonstrations. To improve temporal reasoning, we evaluated recurrent BC variants in which the policy processes a sequence of observations through an LSTM:

$$\hat{a}_t = \pi_\theta(o_{t-L+1:t}), \tag{3}$$

where L is the sequence length.

B. Diffusion Policy

Diffusion Policy models action generation as a conditional denoising diffusion process. Instead of directly predicting a single action, the model iteratively denoises a noisy action sequence conditioned on recent observations. A simplified objective is:

$$\mathcal{L}_{\mathrm{diff}} = \mathbb{E}_{(o,a)\sim\mathcal{D},\epsilon\sim\mathcal{N}(0,I),k}\left[\|\epsilon - \epsilon_\theta(o, a_k, k)\|_2^2\right], \tag{4}$$

where a_k denotes an action sequence corrupted with noise at diffusion step k, and ϵ_θ is the learned noise predictor.

Compared to standard BC, Diffusion Policy can represent multi-modal action distributions and generate coherent action sequences. However, it is computationally more expensive because inference requires multiple denoising steps, and it generally requires more tuning.

V. Behavioral Cloning Experiments

A. Initial BC Baseline

The first experiment used the default feedforward BC template:

- learning rate: 1×10^{-4};
- batch size: 100;
- sequence length: 1;
- epochs: 2000;
- architecture: feedforward MLP;
- observations: low-dimensional state features only.

This configuration achieved a rollout success rate of **0.02**, corresponding to approximately one successful episode out of 50 rollouts. This poor performance is consistent with the limitations of feedforward BC on long-horizon manipulation tasks: the policy has no memory of previous states and cannot reliably infer the current phase of the task.

B. Image-Based BC Baseline

We also evaluated a feedforward image-based BC model that used `agentview_image` and `robot0_eye_in_hand_image` in addition to low-dimensional observations. This configuration achieved a success rate of **0.00**. With only 59 demonstrations, the visual BC model did not learn a useful representation from raw pixels. The compact low-dimensional state representation was easier for BC to optimize.

C. BC Hyperparameter Tuning

Table II summarizes all BC configurations explored during hyperparameter tuning.

TABLE II
BEHAVIORAL CLONING HYPERPARAMETER TUNING RESULTS. BOLD INDICATES THE BEST-PERFORMING CONFIGURATION.

Configuration	LR	Batch	Seq.	Epochs	Model	Success
Original BC	1×10^{-4}	100	1	2000	MLP	0.02
Image BC	1×10^{-4}	64	1	500	MLP + RGB	0.00
High-LR BC	3×10^{-4}	64	1	1000	MLP	0.00
GMM BC	1×10^{-4}	100	1	1000	GMM	0.20
RNN BC (seq 10)	1×10^{-4}	32	10	1000	LSTM	0.30
RNN BC (seq 10, low LR)	3×10^{-5}	32	10	1000	LSTM	0.10
RNN-GMM BC	1×10^{-4}	32	10	1000	LSTM+GMM	0.00
RNN BC (seq 20)	$\mathbf{1\times10^{-4}}$	**32**	**20**	**1000**	**LSTM**	**0.80**
RNN BC (seq 20, 1500 epochs)	1×10^{-4}	32	20	1500	LSTM	0.50
RNN BC (seq 30)	1×10^{-4}	32	30	1000	LSTM	0.50

D. BC Training Curves

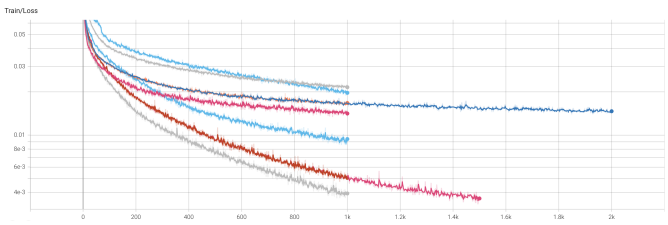

Fig. 2. Behavioral Cloning training loss curves for selected configurations.

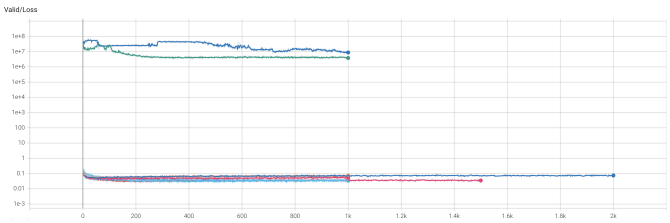

Fig. 3. Behavioral Cloning validation loss curves for selected configurations.

Fig. 4. BC Train and Validation Loss Legend.

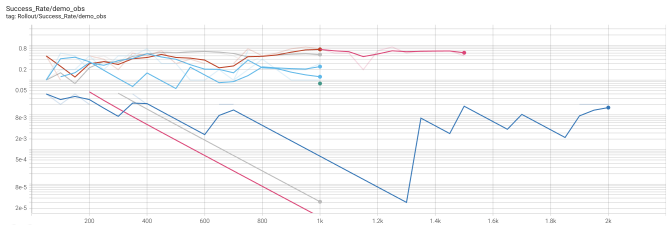

Fig. 5. Rollout success rate over training for the main BC configurations.

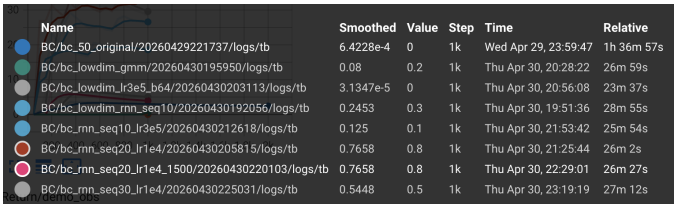

Fig. 6. BC Success Rate Legend.

E. BC Analysis

a) Effect of Temporal Context: The most important improvement came from adding observation history through an LSTM. The `PickPlaceCan` task contains distinct phases: reaching, grasping, lifting, transporting, and releasing. A feedforward policy with sequence length 1 observes only the current state, while an LSTM can use a sequence of past observations to infer the task phase. Increasing the sequence length from 1 to 10 improved success from 0.02 to 0.30, and increasing it to 20 improved success to 0.80.

b) Optimal Sequence Length: The sequence length study showed a non-monotonic trend. Sequence length 20 was best, while sequence length 30 reduced performance to 0.50. This suggests that more temporal context is useful only up to a point; longer histories may add irrelevant information or make optimization more difficult.

c) Effect of Training Duration: Training the sequence length 20 model for 1500 epochs reduced success from 0.80 to 0.50. This suggests that additional supervised training did not improve closed-loop policy quality and may have caused overfitting to the demonstration trajectories.

d) Learning Rate Sensitivity: Increasing the learning rate to 3×10^{-4} produced 0.00 success, suggesting unstable optimization or poor generalization. Reducing the learning rate to 3×10^{-5} for the RNN seq10 model produced only 0.10 success, suggesting underfitting. The best recurrent BC result used 1×10^{-4}.

e) GMM Action Head: GMM BC improved feedforward BC from 0.02 to 0.20, indicating that a multimodal action distribution helped somewhat. However, combining GMM with the LSTM produced 0.00 success, likely because the combined model was too complex for the dataset size or required additional tuning.

f) Best BC Configuration: The best BC configuration was:

- Model: LSTM Behavioral Cloning;
- learning rate: 1×10^{-4};
- batch size: 32;
- sequence length / RNN horizon: 20;
- LSTM hidden dimension: 400;
- LSTM layers: 2;
- epochs: 1000;
- observations: low-dimensional state features;
- **success rate: 0.80**.

VI. Diffusion Policy Experiments

A. Baseline Diffusion Configuration

The original diffusion policy template used:

- optimizer: AdamW;
- learning rate: 1×10^{-4};
- batch size: 100;
- epochs: 2000;
- observation horizon: 2;
- action horizon: 8;
- prediction horizon: 16;
- DDPM training timesteps: 100;
- DDPM inference timesteps: 100;
- rollout evaluation: 50 episodes per evaluation with video rendering.

This template run was substantially slower than the tuned variants. It ran for many hours because of the large number of epochs, large rollout count, video rendering, and 100 DDPM denoising steps at inference. Therefore, due to time constraints, we had to terminate this initial run, in order to run the final comparison using tuned diffusion configurations that completed within the available compute budget.

B. Diffusion Hyperparameter Tuning

Table III summarizes the diffusion configurations evaluated. All tuned configurations used train/validation split keys and TensorBoard logging. The rollout count for tuning was reduced to 10 to make experiments feasible.

TABLE III
DIFFUSION POLICY HYPERPARAMETER TUNING RESULTS. BOLD ENTRIES INDICATE THE BEST-PERFORMING CONFIGURATIONS.

Configuration	Obs.	LR	Batch	Pred. Hor.	Epochs	Success
diffusion_lowdim_base	Low-dim	1×10^{-4}	64	16	1000	0.00
diffusion_lowdim_h32	Low-dim	1×10^{-4}	32	32	1000	0.40
diffusion_image_base	Low+RGB	$\mathbf{1\times10^{-4}}$	**16**	**16**	**500**	**0.50**
diffusion_image_1000_lr1e4	Low+RGB	1×10^{-4}	16	16	1000	0.30
diffusion_image_1000_lr5e5	Low+RGB	$\mathbf{5\times10^{-5}}$	**16**	**16**	**1000**	**0.50**
Template original	Low-dim	1×10^{-4}	100	16	2000	Not completed

C. Invalid Diffusion Configuration Attempt

We also attempted an image-based longer-horizon configuration, `diffusion_image_h32_obs4`, which increased the observation horizon to 4 and prediction horizon to 32. This run failed during rollout with a conditioning feature dimension mismatch:

```
RuntimeError: mat1 and mat2 shapes cannot be
    multiplied (1x558 and 860x512)
```

This configuration is therefore excluded from the final performance table. The failure suggests that increasing the observation horizon in this robomimic diffusion setup can require additional configuration changes to keep the conditioning encoder and rollout observation history consistent.

D. Diffusion Training Curves

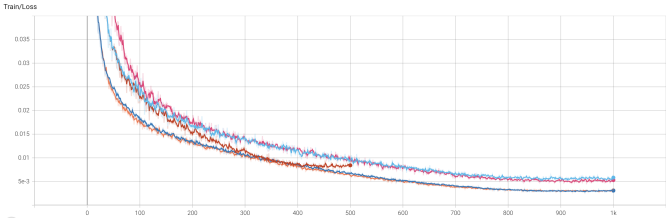

Fig. 7. Diffusion Policy training loss curves for selected configurations.

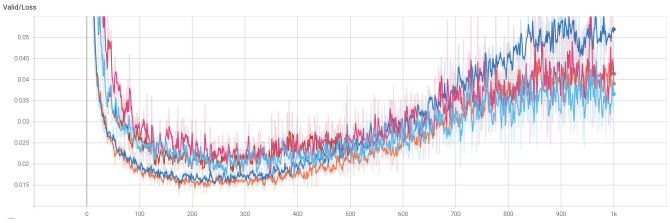

Fig. 8. Diffusion Policy validation loss curves for selected configurations.

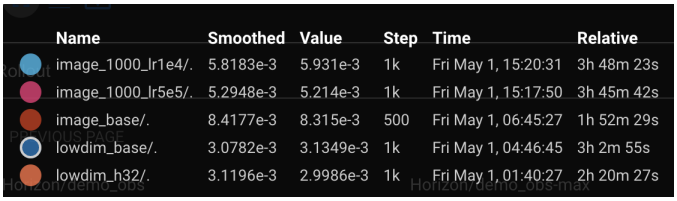

Fig. 9. Diffusion Policy Train and Validation Loss Legend.

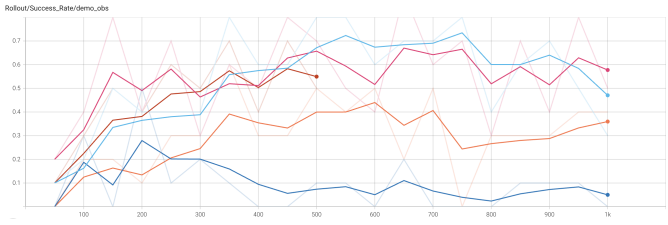

Fig. 10. Rollout success rate over training for the main Diffusion Policy configurations.

E. Diffusion Analysis

a) Low-Dimensional Baseline: The low-dimensional diffusion baseline achieved 0.00 success. This shows that the default low-dimensional diffusion configuration was not sufficient for this dataset and task. Increasing the prediction horizon from 16 to 32 improved low-dimensional diffusion success to 0.40. This suggests that longer action-sequence prediction helped the model produce more coherent multi-step behavior.

b) Image Observations: The best diffusion configurations used both low-dimensional and RGB observations. Both `diffusion_image_base` and `diffusion_image_1000_lr5e5` achieved 0.50 success. This differs from BC, where image observations did not help. Diffusion Policy may have benefited more from visual context because it predicts action sequences rather than single actions.

c) Learning Rate and Training Duration: Increasing image diffusion training from 500 to 1000 epochs at learning rate 1×10^{-4} reduced success from 0.50 to 0.30. Lowering the learning rate to 5×10^{-5} restored success to 0.50. This suggests that longer image-diffusion training required a smaller learning rate to avoid degrading closed-loop policy performance.

d) Best Diffusion Configuration: The best diffusion configurations were:

- `diffusion_image_base`: low-dimensional + RGB observations, learning rate 1×10^{-4}, batch size 16, prediction horizon 16, 500 epochs, success rate 0.50;
- `diffusion_image_1000_lr5e5`: low-dimensional + RGB observations, learning rate 5×10^{-5}, batch size 16, prediction horizon 16, 1000 epochs, success rate 0.50.

VII. Comparison of BC and Diffusion Policy

Table IV compares the best BC and diffusion results.

TABLE IV
Best Behavioral Cloning and Diffusion Policy results.

Method	Best Configuration	Observations	Success	Key Factor
Behavioral Cloning	LSTM seq 20	Low-dim	**0.80**	Temporal context via LSTM
Diffusion Policy	Image diffusion	Low-dim + RGB	0.50	Visual observations + sequence prediction

The best BC model outperformed the best diffusion model. The BC LSTM seq20 policy achieved 0.80 success, while the best diffusion policies achieved 0.50. This suggests that, for this dataset size, the lower-capacity recurrent BC model was easier to optimize and generalized better.

Several factors likely contributed to this result. First, the dataset contains only 59 demonstrations, which may be insufficient for a diffusion model to learn a robust denoising process. Second, the best BC policy used compact low-dimensional observations, reducing the input dimensionality. Third, the LSTM provided temporal memory, which was well suited to the multi-phase nature of `PickPlaceCan`.

Diffusion Policy was still competitive. In particular, image-based diffusion substantially outperformed low-dimensional diffusion, suggesting that visual context helped the generative action-sequence model. However, the diffusion models required more careful tuning and were computationally more expensive.

VIII. Comparison of BC and Diffusion Policy

A. Experimental Design

To study data efficiency, we retrained the best-performing Behavioral Cloning architecture and the best-performing Diffusion Policy architecture using restricted subsets of the demonstration dataset. The goal was to evaluate how the number of available demonstrations affects rollout success rate for both methods.

For Behavioral Cloning, we used the best architecture found in the main hyperparameter search: an LSTM BC policy with sequence length 20, learning rate 1×10^{-4}, batch size 32, and low-dimensional observations. For Diffusion Policy, we used the best practical diffusion setup: image-based Diffusion Policy using both low-dimensional state features and RGB images, with learning rate 1×10^{-4}, batch size 16, observation horizon 2, action horizon 8, and prediction horizon 16.

To make the comparison controlled, we created deterministic subset masks from the same dataset:

- 5 demonstrations,
- 10 demonstrations,
- 20 demonstrations,
- 50 demonstrations.

The same validation mask was used for all subset experiments. Table V reports the rollout success rates obtained for both methods using different dataset sizes.

TABLE V
Effect of the number of demonstrations on rollout success rate.

Demonstrations	BC Success	Diffusion Success
5	0.10	0.00
10	0.10	0.10
20	0.10	0.60
50	0.80	0.90

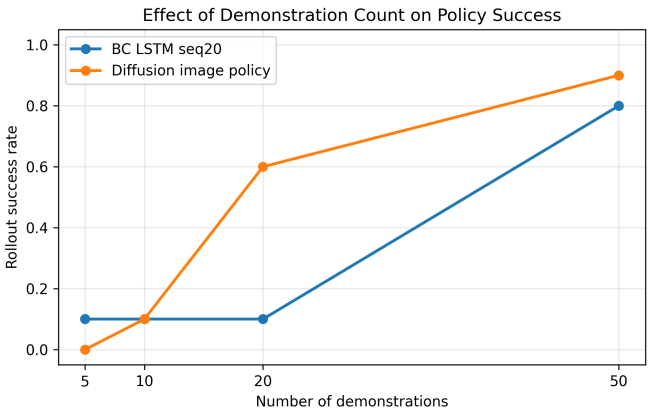

Fig. 11. Rollout success rate as a function of the number of training demonstrations for Behavioral Cloning and Diffusion Policy.

B. Discussion

The demonstration-count experiment shows that both methods were limited when trained with very small datasets. With only 5 demonstrations, BC achieved 0.10 success and Diffusion Policy achieved 0.00 success. With 10 demonstrations, both methods achieved 0.10 success. These low values indicate that neither method had enough state-action coverage to reliably learn the full reaching, grasping, lifting, transporting, and placing sequence.

The methods diverged more clearly as the number of demonstrations increased. With 20 demonstrations, BC still achieved only 0.10 success, while Diffusion Policy improved substantially to 0.60. This suggests that the image-based diffusion model was able to use the additional demonstrations more effectively than the recurrent BC model in this intermediate-data regime. With 50 demonstrations, BC improved sharply to 0.80, while Diffusion Policy reached 0.90, the best success rate observed in the controlled demonstration-count study.

These results suggest that BC exhibited a threshold-like behavior: performance stayed low for 5, 10, and 20 demonstrations, then improved strongly once 50 demonstrations were available. In contrast, Diffusion Policy improved more smoothly as the number of demonstrations increased, especially from 10 to 20 and from 20 to 50 demonstrations. Based on this trend, Diffusion Policy appeared more stable with respect to increasing dataset size in the subset experiment, while BC was more sensitive to whether enough demonstrations were available to cover the full task distribution. The final 50-demonstration result also shows that image-based Diffusion Policy can outperform the best recurrent BC policy when enough demonstrations are provided.

IX. Challenges

A. Cluster and Rendering Setup

MuJoCo rendering did not work reliably on cluster login nodes. Rendering-dependent operations, including observation generation and rollout video production, required GPU nodes with EGL rendering enabled using `MUJOCO_GL=egl`. Demonstration collection was performed locally because keyboard control and interactive rendering were more stable.

B. Dependency and Version Compatibility

Version compatibility was important. A mismatch between the robosuite version used during data collection and the robosuite version installed on the cluster caused controller configuration errors during dataset conversion. This was resolved by installing a compatible robosuite version on the cluster.

C. BC Sensitivity to Architecture

BC performance was highly sensitive to sequence length and architecture. Switching from feedforward MLP BC to LSTM BC with sequence length 20 improved success from 0.02 to 0.80. However, increasing sequence length to 30 or training for 1500 epochs reduced success to 0.50, demonstrating that larger models or longer training do not necessarily improve closed-loop performance.

D. Diffusion Policy Runtime

Diffusion Policy training was substantially more expensive than BC. The original template used 2000 epochs, batch size 100, 50 rollout episodes per evaluation, and 100 denoising steps, making it very slow. Tuned diffusion configurations with smaller batch sizes, fewer epochs, and 10 rollout episodes per evaluation completed much faster while producing better or comparable success rates.

X. Conclusion

This paper presented a comparative study of Behavioral Cloning and Diffusion Policy for the robosuite `PickPlaceCan` task. We collected 59 successful expert demonstrations using a Panda robot and trained multiple imitation learning policies with robomimic. The initial feedforward BC baseline achieved only 0.02 rollout success, showing that a memoryless policy was not sufficient for this multi-stage manipulation task. The most important improvement for BC was the addition of temporal context through an LSTM policy. Increasing the sequence length from 1 to 10 improved success to 0.30, and increasing it further to 20 improved success to 0.80. However, the effect was not monotonic: increasing the sequence length to 30 or training the sequence length 20 model for 1500 epochs reduced success to 0.50, suggesting that excessive temporal context or longer training can degrade closed-loop performance.

Diffusion Policy was more computationally expensive and more sensitive to hyperparameter choices, but it ultimately achieved the highest success rate in the controlled demonstration-count experiment. The low-dimensional diffusion baseline achieved 0.00 success, while increasing the prediction horizon improved low-dimensional diffusion to 0.40. Image-based diffusion policies performed better, with the earlier image-based diffusion runs reaching 0.50 success. In the final demonstration-count study, the image-based Diffusion Policy trained with 50 demonstrations achieved 0.90 success, outperforming the best BC model at 0.80. The demonstration-count results showed that BC had a threshold-like behavior,

remaining at 0.10 success for 5, 10, and 20 demonstrations before improving to 0.80 at 50 demonstrations. Diffusion Policy improved more gradually, reaching 0.60 with 20 demonstrations and 0.90 with 50 demonstrations. Overall, the experiments suggest that recurrent BC is a strong and efficient baseline when enough low-dimensional demonstrations are available, while image-based Diffusion Policy can achieve superior final performance and appears more stable as dataset size increases.

REFERENCES

[1] ARISE Initiative, "robosuite: A Modular Simulation Framework and Benchmark for Robot Learning," GitHub repository. Available: https://github.com/ARISE-Initiative/robosuite

[2] ARISE Initiative, "robomimic: A Framework for Robot Learning from Demonstrations," GitHub repository. Available: https://github.com/ARISE-Initiative/robomimic